

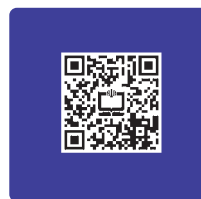
برنامه‌نویسی پایتون (۲)



سال گذشته در پودمان برنامه‌نویسی آموختید که چگونه با زبان برنامه‌نویسی پایتون، یک برنامه ساده رایانه‌ای بنویسید و شکل‌های هندسی منتظم را طراحی کنید. در این پودمان با مفاهیم آرایه‌ها، توابع، ماژول، مدیریت استثنا و فایل‌ها در برنامه‌نویسی به زبان پایتون آشنا خواهید شد. با یادگیری و تسلط روی این مهارت‌ها، قادر به نوشتن برنامه‌های زیبا و کاربردی و انجام برخی از تکالیف درسی خود خواهید شد.

برخی از شایستگی‌هایی که در این پودمان به‌دست می‌آورید :

- کار گروهی، مسئولیت‌پذیری، مدیریت منابع، فناوری اطلاعات و ارتباطات و اخلاق حرفه‌ای؛
- کسب مهارت‌های حل مسئله و پرورش تفکر الگوریتمی؛
- توانایی کار با آرایه‌ها در پایتون؛
- توانایی کار با توابع در پایتون؛
- توانایی ساختن فایل پایتونی (ماژول)؛
- توانایی مدیریت استثناها در پایتون؛
- توانایی ذخیره‌سازی فایل‌های پایتونی در حافظه دائمی رایانه؛
- توانایی استفاده کاربردی از زبان برنامه‌نویسی پایتون برای حل مسئله‌های ریاضی، علوم تجربی و زندگی روزمره.



قبل از شروع قسمت دوم برنامه‌نویسی پایتون، با اسکن رمزینه روبه‌رو، فیلم مروری بر آنچه را که در سال گذشته خواندید، مشاهده کنید.

در سال گذشته با مباحثی همچون متغیرها، عملگرها، ساختارهای شرطی if-elif، if-else و حلقه یا همان ساختار تکرار for آشنا شدید. افزون بر این‌ها، چگونگی دریافت مقادیر از ورودی و چاپ مقدار متغیرها در خروجی را دیدید. همان‌طور که می‌دانید متغیرها، خانه‌ای از حافظه اند که مقداری را به‌طور موقت در خود ذخیره می‌کنند. از این‌رو برنامه‌نویسان با دو پرسش اساسی مواجه می‌شوند.

۱ آیا می‌توان یک ساختار داده‌ای را ایجاد کرد که چندین مقدار را داخل خودش ذخیره کند؟

۲ چگونه می‌توان داده‌های ورودی از کاربر را به صورت دائمی در حافظه رایانه ذخیره کرد؟



برای پاسخ به پرسش اول باید از ساختارهای آرایه‌ای استفاده کرد و در پاسخ به پرسش دوم باید متغیرهای برنامه را روی حافظه دائمی رایانه مثل هارد دیسک ذخیره کرد. به همین دلیل در این درس به بررسی مبحث آرایه‌ها و فایل‌ها می‌پردازیم. همچنین از مفهوم تابع برای کاهش تعداد خطوط برنامه و خوانایی بهتر آن استفاده خواهیم کرد. در ضمن می‌توان یک فایل پایتونی (ماژول)، ایجاد کرده و داخل آن مواردی مانند تابع (function)، فهرست (list)، چندتایی (tuple)، مجموعه (set) و دیکشنری (dictionary) را نوشت و در سایر برنامه‌های دیگر از آن ماژول استفاده کرد.

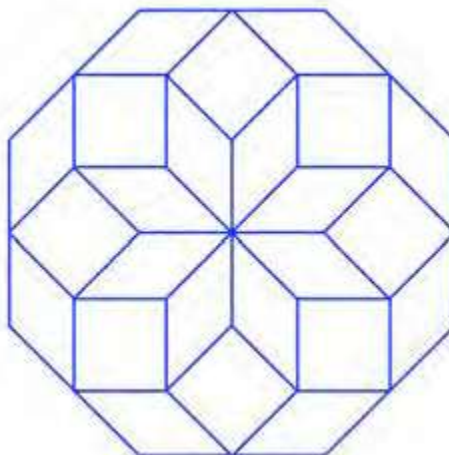
کار کلاسی

قطعه برنامه زیر را که توسط کتابخانه turtle نوشته شده است، در محیط IDLE بنویسید. خروجی آن به شکل زیر است:

```

import turtle
t=turtle.Turtle()
t.shape("turtle")
t.color("blue")
t.speed("fast")
screen = turtle.Screen()
t.width(3)
for i in range(8):
    for j in range(8):
        t.forward(100)
        t.right(45)
        t.right(45)
t.hideturtle()

```



برنامه‌ای بنویسید که تعداد ضلع‌های یک شکل هندسی و تعداد تکرار آن را از ورودی دریافت کرده و سپس طرح را به صورت کامل ترسیم کند. نتیجه را به کلاس درس ارائه دهید.

(راهنمایی: برنامه را با استفاده از متد textinput بنویسید. به این منظور از کتابخانه turtle استفاده کنید.)

آرایه‌ها

برای ذخیره چندین داده در یک متغیر از ساختاری به نام آرایه (collection) استفاده می‌شود. در زبان پایتون، چهار نوع آرایه به نام‌های لیست، تاپل، ست و دیکشنری داریم. از میان این‌ها، هر چند عملکرد آرایه‌های لیست و تاپل شباهت زیادی به یکدیگر دارند، ولی پس از تعریف لیست، عناصر آن را می‌توان تغییر داد. اما در مورد تاپل این کار امکان‌پذیر نیست. دو آرایه دیگر، یعنی ست و دیکشنری نیز به هم شبیه‌اند. با این تفاوت که در دیکشنری، هر کدام از عناصر (آیتم‌ها) از ترکیب کلید: مقدار (key:value) ایجاد می‌شوند. اما در آرایه ست، عناصر فقط دارای مقدار (value) هستند.



با اسکن رمزیننه بالا
با میحت آرایه‌ها و
تمرین‌های مرتبط با
آن آشنا شوید.

ویژگی‌های عناصر تاپل

■ غیر قابل تغییرند. ■ شماره گذاری می‌شوند.

■ داده تکراری می‌پذیرند. ■ دارای ترتیب‌اند.

مثال:

`myTuple=(16,'karim',20,16,7.5,12,False)`

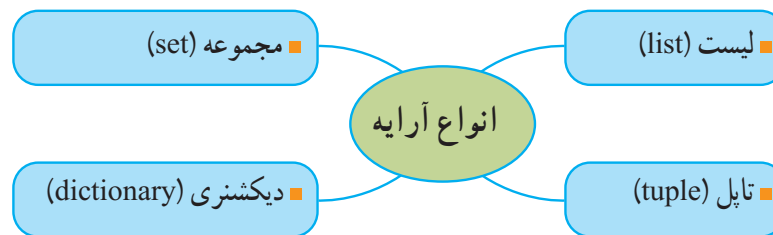
ویژگی‌های عناصر لیست

■ قابل تغییرند. ■ شماره گذاری می‌شوند.

■ داده تکراری می‌پذیرند. ■ دارای ترتیب‌اند.

مثال:

`myList=[10,20,'omid',30,10,True,45]`



با اسکن رمزیننه بالا
کار با میحت لیست
و تمرین‌های مرتبط
به آن را فرا بگیرید.

ویژگی‌های عناصر دیکشنری

■ دارای ترتیب هستند (از پایتون نسخه ۳/۷ به بعد)

■ عناصر قابل تغییرند.

■ داده تکراری می‌پذیرند. با این شرط که آخرین مقدار برای آن کلید در نظر گرفته شود.

■ به صورت ترکیبی از key:value هستند. اگر کلید تکراری داشته باشیم، آخرین مورد را می‌پذیرد.

مثال:

`myDict={'name':'omid','family':'rezaee','age':30}`

ویژگی‌های عناصر مجموعه

■ دارای ترتیب نیستند.

■ غیر قابل تغییرند.

■ داده تکراری نمی‌پذیرند.

■ در صورت تکراری بودن عناصر، فقط یک مورد را ذخیره می‌کنند.

مثال:

`mySet={10,15,'ali',11,63,14}`

دسترسی به عناصر لیست

اگر مبنای شماره گذاری عناصر لیست از **راست به چپ** باشد. باید از ۰ شروع کنیم. مثال :

```
myList=[12,15,"omid",8,43,True,12,36,9]
```

۱۲	۱۵	'omid'	۸	۴۳	True	۱۲	۳۶	۹
-۹	-۸	-۷	-۶	-۵	-۴	-۳	-۲	-۱

myList [-7] → "omid" myList[-8]→15

اگر مبنای شماره گذاری عناصر لیست از **چپ به راست** باشد، باید از صفر شروع کنیم. مثال :

```
myList = [12,15,23,8,43,560,12,36,9]
```

۱۲	۱۵	۲۳	۸	۴۳	۵۶۰	۱۲	۳۶	۹
۰	۱	۲	۳	۴	۵	۶	۷	۸

myList[2] → 23 myList [5] → 560

عملیات روی لیست‌ها

در پایتون، عملیات روی عناصر لیست‌ها عبارت‌اند از : اضافه کردن، حذف، مرتب‌سازی و پیمایش عناصر.

مرتب‌سازی عناصر لیست

با `sort` می‌توان عناصر لیست را به‌طور صعودی یا نزولی مرتب کرد.

```
thisList=['orange','mango','kiwi','banana']
thisList.sort()
print(thisList)
```

خروجی قطعه برنامه بالا یک لیست به شکل زیر است که به ترتیب حروف الفبای انگلیسی مرتب شده است :

```
['banana','kiwi','mango','orange']
```

همچنین برای مرتب‌سازی به ترتیب نزولی از روش زیر استفاده می‌کنیم :

```
thisList=["orange","mango","kiwi","banana"]
thisList.sort(reverse=True)
print(thisList)
```

```
['orange','mango','kiwi','banana']
```



با اسکن رمزینه بالا کار با لیست‌ها را فراگیرید.

اضافه کردن عنصر به لیست

myList=[10,20,45]

برای اضافه کردن یک عنصر به انتهای لیست بالا از `append` استفاده می‌شود.

```
myList.append(50)
print(myList)
```

به عنوان مثال :

خروجی برنامه بالا به شکل زیر است :

```
[10,20,45,50]
```

برای درج عنصر به یک مکان خاصی در لیست از `insert` استفاده می‌کنیم. به عنوان مثال برای اضافه کردن مقدار ۱۰۰ در مکان شماره ۲ از دستور زیر استفاده می‌شود :

```
myList.insert(2,100)
[10,20,100,45,50]
```



با اسکن رمزینه بالا کار با مبحث اضافه کردن عنصر به لیست را فراگیرید.

پیمایش عناصر لیست

پیمایش عناصر لیست به معنای حرکت روی عناصر است.

فرض کنید که یک لیست به شکل زیر داریم:

```
myList=[100,213,350,625,140]
```

```
for i in myList:
```

```
    print(i,end=' ')
```

در قطعه برنامه بالا توسط یک حلقه for روی عناصر لیست حرکت کرده و آن‌ها را در یک سطر چاپ می‌کنیم. در واقع اینجا i به معنی عناصر لیست است.

```
100 213 350 625 140
```

تمرین: چگونه می‌توان توسط یک ساختار شرطی در داخل حلقه for عناصر زوج را در لیست بالا نمایش داد؟



با اسکن رمزینه رویه رو، کار با مبحث پیمایش عناصر لیست را به همراه تمرین‌های مرتبط با آن فرا بگیرید.

حذف کردن عنصر از لیست

برای حذف عنصر از لیست، می‌توان از متدهای pop و remove و clear استفاده کرد.

مثال: فرض کنید که یک لیست به شکل زیر داریم:

```
myList=[10,20,45,"book",80,45,30]
```

برای حذف یک عنصر از انتهای لیست، از متد pop به شکل زیر استفاده می‌شود:

```
myList.pop()
```

حال اگر لیست را دوباره توسط دستور print چاپ کنیم، لیست به شکل [10,20,45,"book",80,45] تبدیل می‌شود.

همچنین برای حذف عنصر از مکان شماره ۱ لیست بالا، می‌توان از متد pop به همراه شماره خانه استفاده کرد.

```
myList.pop(1)
```

بنابراین عدد ۲۰ از لیست بالا حذف شد. [10,45,"book",80,45]
برای حذف عنصر ۴۵ (اولین عدد ۴۵) از لیست بالا، از دستور زیر استفاده می‌شود.

```
myList.remove(45)→ [10,"book",80,45]
```

مقایسه بین لیست و تاپل

جدول ۳-۱

چند تایی (tuple)		لیست (list)	
انتساب ^۱	myTuple = (10, 15, 36, 15) myTuple [2] = 100 → Error print(myTuple)	انتساب	myList = [10, 15, 36, 15] myList [2] = 100 print(myList) → [10, 15, 100, 15]
محدوده ^۲	myTuple = (10, 15, 36, 15) print(myTuple [1: 3]) → (15, 36) print (myTuple [: 2]) → (10, 15)	محدوده	myList = [10, 15, 36, 15] print(myList [1: 3]) → [15, 36] print (myList [1:]) → [15, 36, 15]
حذف ^۳	myTuple = (10, 15, 36, 15) myTuple.remove(15) → Error	حذف	myList = [10, 15, 36, 15] myList.remove(15) print(myList) → [10, 36, 15]
باز کردن ^۴	fruits = ("apple", "banana", "cherry") (green , yellow , red) = fruits print(green , yellow , red) → apple banana cherry	باز کردن	fruits = ["apple", "banana", "cherry"] [green , yellow , red] = fruits print(green , yellow , red) → apple banana cherry



با اسکن رمزینه رویه رو، کار با مبحث لیست و تاپل و تمرین‌های مرتبط با آن را فرا بگیرید.

نکته: در دستور `print(myList[3])`، ابتدای محدوده مشخص نیست. در این مواقع پایتون آغاز محدوده را از اولین عنصر لیست در نظر می‌گیرد و دستور به صورت `print(myList[0:3])` تبدیل می‌شود.

همچنین در دستور `print(myList[2:])` به دلیل آنکه انتهای محدوده مشخص نیست، پایتون از عنصر شماره ۲ تا انتهای لیست را چاپ می‌کند.

نکته: با توجه به جدول ۱-۳، عملیات حذف و انتساب در آرایه تاپل امکان‌پذیر نیست. زیرا تاپل، به‌طور ذاتی قابل تغییر نیست، ولی می‌توان آن را به لیست تغییر داد. پس از آن، تغییرات لازم مثل حذف و انتساب را روی آن انجام داده و دوباره آن را به تاپل تغییر می‌دهیم.

مثال: تغییر عناصر تاپل

```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)
print(x)
```

در کدهای بالا، ابتدا (در خط دوم) تاپل `x` به لیست `y` تبدیل شده است و سپس (در خط سوم) مقدار عنصر شماره ۱ را تغییر دادیم، آنگاه دوباره در خط چهارم آن را به تاپل تبدیل کرده و سرانجام در خط پنجم آن را چاپ کردیم.

خروجی برنامه: `("apple", "kiwi", "cherry")`

مثال: حذف عنصر از تاپل

```
thisTuple = ("apple", "banana", "cherry")
thisList = list(thisTuple)
thisList.remove("apple")
thisTuple = tuple(thisList)
print(thisTuple)
```

در کدهای بالا، ابتدا (در خط دوم) تاپل `thisTuple` به لیست `thisList` تبدیل شده، سپس (در خط سوم)، عنصر `apple` را حذف کردیم، آنگاه دوباره در خط چهارم آن را به تاپل تبدیل کرده و در نهایت در خط پنجم آن را چاپ کردیم.

خروجی برنامه: `("banana", "cherry")`

متدهای قابل اجرا روی لیست و تاپل

در جدول ۲-۳ با انواع متدهای قابل اجرا روی لیست‌ها و چندتایی‌ها آشنا شده و تفاوت‌های آن‌ها را با هم مقایسه می‌کنیم.

جدول ۲-۳ — متدهایی که روی لیست و تاپل انجام می‌شود

چندتایی	لیست	متد
تعداد عناصر تاپل را شمارش می‌کند myTuple = (10,20,35,62,75) print(len(myTuple)) → 5	تعداد عناصر لیست را شمارش می‌کند myList = [10,20,35,62,75] print(len(myList)) → 5	len تعداد عناصر آرایه
thisTuple = (12,16,10,9,23) print(min(thisTuple)) → 9	thisList = [12,16,10,9,23] print(min(thisList)) → 9	min کمترین مقدار عناصر
thisTuple = (12,16,10,9,23) print(max(thisTuple)) → 23	thisList = [12,16,10,9,23] print(max(thisList)) → 23	max بیشترین مقدار عناصر
fruits = ('apple', 'banana', 'cherry') fruits.reverse() print(fruits) → Error	fruits = ['apple', 'banana', 'cherry'] fruits.reverse() print(fruits) → ['cherry', 'banana', 'apple']	reverse معکوس کردن لیست

مثال: برنامه‌ای بنویسید که ۵ عدد از ورودی دریافت کند و به انتهای آرایه لیست اضافه و در خروجی، عناصر فرد را چاپ کند.

```
myList=[]
for i in range(5):
    myList.append(int(input(f'Enter Number {i+1}: ')))
for i in myList:
    if i % 2 != 0:
        print(i, end=' ')
```



تحقیق کنید که کاراکتر f در خط سوم قطعه برنامه بالا، چه کاربردی دارد؟

مثال: نحوه تعریف دیکشنری

به قطعه برنامه زیر دقت کنید:

```
thisdict={
    "brand": "دنا",
    "model": "Turbo",
    "year": 1401
}
```

```
print(thisdict)
```

در برنامه بالا، ابتدا یک دیکشنری با سه آیتیم تعریف کردیم. هر آیتیم دارای یک کلید: مقدار است. سپس توسط دستور `print(thisdict)` تمام آیتیم‌ها را چاپ می‌کنیم و خروجی به شکل زیر است:

```
{'brand': 'دنا', 'model': 'Turbo', 'year': 1401}
```

مثال: نحوه تعریف ست

اگر مجموعه‌ای از اسامی دانش‌آموزان را داشته باشیم، هر بار که آن را توسط دستور `print` چاپ می‌کنیم، خروجی‌ها از لحاظ ترتیب متفاوت خواهد بود. علت این است که آرایه ست فاقد ترتیب است:

```
mySet={'amin', 'parnian', 'arian', 'Ehsan'}
print(mySet)
```

با اجرای برنامه بالا، خروجی به صورت زیر است:

```
{'amin', 'parnian', 'arian', 'Ehsan'}
```

برنامه بالا را چندین بار اجرا کنید و خروجی‌ها را با هم مقایسه کنید. خواهید دید که هر بار خروجی متفاوت می‌شود.

با اسکن رمزینه روبه‌رو کار با مبحث ست و دیکشنری را به همراه تمرین‌های مرتبط با آن فرا بگیرید.



مفهوم تابع

به قطعه برنامه‌ای که یک بار نوشته شده و نامی دلخواه برای آن انتخاب می‌شود، تابع می‌گویند. تابع را می‌توان هر چند بار که لازم باشد فراخوانی کرد. این کار باعث کمتر شدن تعداد خطوط برنامه و راحت‌تر شدن کار برنامه‌نویسی می‌شود.

نکته: خروجی متدها توسط دستور `return` برگشت داده می‌شود.

مثال: برنامه‌ای بنویسید که تابعی با سه ورودی ایجاد کند و حاصل جمع آن‌ها را با دستور `return` برگرداند.

```
File Edit Format Run Options Window Help
1 def myFunction(x,y,z):
2     return x+y+z
3
4 m=int(input('Enter m:'))
5 n=int(input('Enter n:'))
6 p=int(input('Enter p:'))
7 print('sum=',myFunction(m,n,p))
```

روش تعریف تابع

```
Enter m:10
Enter n:20
Enter p:30
sum= 60
```

خروجی برنامه

توضیح: در قطعه برنامه بالا با استفاده از دستور `def` یک تابع در خط ۱ به نام `myFunction` ایجاد نمودیم و ورودی‌های تابع را با `x` و `y` و `z` مشخص کردیم و توسط دستور `return` در خط ۲ حاصل جمع آن‌ها را محاسبه کردیم و مقدار آن را برگرداندیم. سپس در خطوط ۴ و ۵ و ۶ سه مقدار `m`, `n`, `p` را از ورودی دریافت و آنگاه در خط ۷ تابع را فراخوانی کردیم. توجه کنید که تعداد و ترتیب پارامترها در زمان تعریف تابع با تعداد و ترتیب آرگومان‌ها در زمان فراخوانی یکسان باشد.

آنچه در مورد کار با تابع لازم است بدانید در شکل زیر آمده است.



کار کلاسی

تابعی بنویسید که دو عدد را دریافت کرده و عدد بزرگ‌تر (بیشینه) را چاپ کند.



با اسکن رمزینه روبه‌رو کار با مبحث تابع و تمرین‌های مرتبط با آن را فرا بگیرید.

ماژول

به هر فایل پایتونی با پسوند `.py` ماژول (module) گفته می‌شود. داخل ماژول می‌توان تابع، لیست، تاپل، ست، دیکشنری و ... ایجاد کرد. ماژول نوشته شده را می‌توان در ابتدای هر برنامه‌ای وارد (`import`) و از عناصر داخل آن استفاده کرد. برای این کار از دو روش زیر استفاده می‌کنیم:

الف) دستور `import ...`

ب) دستور `from ... import ...`

در روش الف، هنگام استفاده از عناصر داخل ماژول، لازم است نام ماژول قبل از آن نوشته شود.

در روش ب، هنگام استفاده از عناصر داخل ماژول نیازی به نوشتن نام ماژول قبل از آن نیست.



با اسکن رمزینه بالا کار با مبحث ماژول و تمرین‌های مرتبط با آن را فرا بگیرید.



```

def myFunction(x,y):
    if x>y:
        return x
    else:
        return y
    return x+y

myList=[10,20,30,40,50]
myTuple=('amir','reza','amin','sardogh')
mySet={10,15,46,2,28,90}
Person={
    'name':'kamran',
    'family':'karimi',
    'Age':21
}
    
```

نام تابع

لیست

تاپل

ست

دیکشنری

در شکل روبه‌رو ماژولی به نام myModule نوشته شده است که شامل تابع، لیست، تاپل، ست، و دیکشنری است. این ماژول را به دو روش الف و ب در برنامه دیگری وارد و از عناصر داخل آن استفاده کنید.

وارد کردن ماژول در ابتدای برنامه به دو روش

در ادامه کار کلاسی بالا، دو برنامه زیر را نوشته و خروجی آن‌ها را با هم مقایسه کنید.

منظور از ستاره یعنی همه عناصر داخل ماژول

```

from myModule import *

print(myFunction(10,20))
print(myList[4])
print(myTuple[2])
print(mySet)
print(Person['family'])
    
```

(ب) روش from...import

```

import myModule

print(myModule.myFunction(10,20))
print(myModule.myList[4])
print(myModule.myTuple[2])
print(myModule.mySet)
print(myModule.Person['family'])
    
```

(الف) روش import

در اینجا باید نام ماژول را نوشت

خروجی دو برنامه بالا به صورت یکسان است، ولی از دو روش متفاوت استفاده شده است:

20

50

'amir'

10,15,2,46,90,28

'karimi'

نکته : افزون بر ماژول‌هایی که برنامه‌نویس می‌نویسد، دسته دیگری از ماژول‌های آماده نیز وجود دارند که از قبل در پایتون تعریف شده‌اند و می‌توان از آن‌ها استفاده کرد، مانند ماژول‌های turtle, numpy, pygame... که برای کار با آرایه‌ها و اشکال هندسی و ... به کار می‌روند.

کار کلاسی



آرایه‌هایی که با ماژول numpy ایجاد می‌شوند، از نظر سرعت و کارایی چه تفاوتی با آرایه‌های لیست دارند؟ در گروه خود در این باره بحث و گفت‌وگو کنید و نتیجه را به کلاس ارائه دهید.

کار غیر کلاسی



با اسکن رمزینه بالا کار با مبحث ماژول و تمرین‌های مرتبط با آن را فرا بگیرید.

با استفاده از آنچه در مباحث آرایه‌ها، توابع، ماژول و فایل‌ها آموختید، با اسکن رمزینه روبه‌رو و مشاهده فیلم، با روش ساخت یک بازی پایتونی زیبا، آشنا شوید و مشابه آن را بسازید.

خوب است بدانید*



مفهوم list comprehension

یک روش ساده در پایتون برای افزودن برخی از عناصر یک لیست براساس یک معیار معین به لیست دیگر وجود دارد. با استفاده از این امکان می‌توان در خطوط کدنویسی صرفه‌جویی کرد. این مسئله به‌طور خاص در حلقه‌ها بسیار مهم است. با توجه به شکل‌های الف و ب، دو قطعه برنامه نوشته شده معادل یکدیگرند. در شکل (الف) از روش معمول برای استخراج عناصر زوج list1 و اضافه شدن آن‌ها به list2 استفاده شده است، ولی در شکل (ب) از روش list comprehension استفاده شده است. این برنامه‌ها از list1 اعداد زوج را انتخاب و به list2 اضافه می‌کند. در انتها list2 را چاپ می‌کند که خروجی برنامه به‌صورت زیر دیده می‌شود :

[10,2,56,46]

```
1 list1=[10,2,56,33,46,21]
2 list2=[x for x in list1 if x%2 ==0]
3 print (list2)
```

شکل ب

```
1 list1=[10,2,56,33,46,21]
2 list2=[]
3
4 for x in list1:
5     if x%2==0:
6         list2.append(x)
7
8 print(list2)
```

شکل الف

*این قسمت اختیاری است و می‌تواند تدریس نشود.

۱ دو قطعه برنامه الف و ب را با یکدیگر مقایسه کنید.

۲ فرض کنید یک لیست حاوی تعدادی عدد صحیح داریم. توسط list comprehension عناصر آن را پیمایش کنید. در صورتی که عنصر زوج پیدا شد، در لیست دوم عبارت Even را اضافه و در غیر این صورت عبارت Odd را اضافه کنید. در انتها لیست دوم را چاپ کنید.

۳ تابعی بنویسید که یک لیست شامل تعدادی عدد را دریافت کند و با پیمایش عناصر آن، تعداد عناصر زوج لیست را توسط دستور return برگرداند. سپس تابع را فراخوانی کنید.

۴ تابعی بنویسید که یک لیست شامل اسامی هم کلاسی هایتان را دریافت کرده، با پیمایش عناصر آن، تعداد عناصر لیست که شامل حرف m باشد را توسط دستور return برگرداند. سپس تابع را فراخوانی کنید.



با اسکن رمزینه بالا با این فعالیت بیشتر آشنا شوید.

تابع لامبدا (lambda)



به توابعی که دارای یک یا چند ورودی اند ولی بدنه آن‌ها، همواره شامل یک عبارت است تابع لامبدا گفته می‌شود. رایج‌ترین استفاده از تابع لامبدا در کدهایی است که نیازمند توابع یک خطی ساده‌ای هستند.

در شرایطی که در بدنه تابع بیش از یک عبارت باشد، نمی‌توان از تابع لامبدا استفاده کرد.

توضیح: توابع لامبدا به خودی خود دارای نام نیستند. پس، از متغیرهای x, y, z و w که در خطوط ۱ و ۴ و ۷ و ۱۰ تعریف شده است، به عنوان نام تابع استفاده شده است.

```

1 x = lambda a : a + 10
2 print("x=", x(5))
3
4 y = lambda a, b : a + b
5 print("y=", y(5, 7))
6
7 z = lambda a, b, c : a + b + c
8 print("z=", z(5, 7, 2))
9
10 w = lambda a, b, c, d : a + b + c + d
11 print("w=", w(5, 7, 3, 4))
12
13
14

```

فقط یک عبارت در داخل بدنه متد وجود دارد

یک ورودی دو ورودی سه ورودی چهار ورودی



با اسکن رمزینه بالا کار با تابع لامبدا را فرا بگیرید و با کاربرد آن آشنا شوید.



برخی مواقع در زمان اجرای کدها با وجود اینکه گرامر زبان (syntax) و دستورات را رعایت کرده‌ایم، با حالتی به نام استثنا (Exception) مواجه می‌شویم. برای مثال در هنگام تقسیم دو عدد در صورتی که خارج قسمت صفر باشد، حالت استثنا نامیده می‌شود.

با اسکن رمزینه بالا
کار با مبحث مدیریت
استثنا و تمرین‌های
مرتبط با آن را فرا
بگیرید.

در چنین مواقعی برنامه‌نویس باید با استفاده از روش‌های مدیریت استثنا (Exception Handling) این مسئله را حل نماید.

مثال: برنامه‌ای بنویسید که دو عدد صحیح از ورودی دریافت نماید و هنگام تقسیم دو عدد، حالت استثنا (تقسیم یک عدد بر صفر) را مدیریت کند.

```
1 num1=int(input('Enter number1:'))
2 num2=int(input('Enter number2:'))
3 try:
4     print(num1/num2)
5 except:
6     print("خطای تقسیم بر صفر")
7 else:
8     print("انجام عمل تقسیم")
9 finally:
10    print("پایان اجرای برنامه")
11
```

دستور و یا دستوراتی که فکر می‌کنیم ممکن است دچار استثنا شود در منطقه try می‌نویسیم.
دستورات داخل بخش except زمانی اجرا می‌شود که در بخش try دچار استثنا شده باشیم.
دستورات داخل بخش else زمانی اجرا می‌شود که در بخش try دچار استثنا نشده باشیم.
دستورات داخل بخش finally همیشه اجرا می‌شود چه دچار استثنا شده باشیم چه نشده باشیم.

Enter number1:10
Enter number2:20
0.5
انجام عمل تقسیم پایان اجرای برنامه

Enter number1:10
Enter number2:0
خطای تقسیم بر صفر پایان اجرای برنامه

در اینجا استثنا رخ داده است چون عدد دوم صفر است. در اینجا استثنا رخ نداده است.

فایل‌ها



وقتی برنامه‌ای را در پایتون ذخیره می‌کنیم، کدهای برنامه در حافظه دائمی و مقادیر متغیرها و آرایه‌ها و ... در حافظه موقت یا RAM ذخیره می‌شوند. به همین دلیل وقتی برنامه‌نویس می‌خواهد مقادیر متغیرها و آرایه‌ها و ... را روی حافظه دائمی، مانند هارد دیسک ذخیره کند باید روی هارد رایانه، فایلی ایجاد کرده و داده‌های موقت را داخل آن ذخیره کند.

با اسکن رمزینه بالا
مبحث مدیریت فایل
را فراگیرید.

فایل‌ها دو نوع هستند: فایل‌های متنی و فایل‌های دودویی (باینری)

محتوای فایل‌های متنی با پسوند .txt توسط ویرایشگری مثل notepad قابل خواندن است، ولی برای دیدن محتوای فایل‌های دودویی، لازم به برنامه‌نویسی است و در حالت عادی توسط ویرایشگر notepad قابل خواندن نیستند.

```

File Edit Format Run Options Window Help
1 f = open("d://amir/myFile.txt", "a")
2 f.write("omid is a good student.")
3 f.close()
4
5 f = open("d://amir/myFile.txt", "r")
6 print(f.read())

```

در خط ۱، فایلی به نام myFile.txt را در فولدر amir واقع در درایو D باز کرده ایم و می خواهیم رشته ای را به انتهای آن اضافه کنیم. سپس در خط ۲ رشته omid is a good student را به انتهای فایل اضافه و در خط ۵ دوباره فایل را باز کرده ایم. برای خواندن (r)، ابتدا فایل را باز و سپس آن را خوانده و چاپ می کنیم. "a" مخفف append به معنای اضافه کردن به انتهای فایل است.

```

1 f = open("d://amir/myFile.txt", "w")
2 f.write("All is a Good Student!")
3 f.close()
4
5 #open and read the file after the overwriting:
6 f = open("d://amir/myFile.txt", "r")
7 print(f.read())

```

در خط ۱، فایل متنی myFile.txt که در داخل درایو D ساخته ایم را توسط دستور open برای خواندن (r) باز می کنیم و آن را در یک متغیر به نام f ذخیره می کنیم. سپس در خط ۶ محتویات آن را توسط تابع read خوانده و چاپ می کنیم. "r" مخفف read به معنای خواندن است.



کار کلاسی

دو قطعه برنامه بالا را اجرا و خروجی آن ها را با هم مقایسه کنید.

نکته: گاهی اوقات ممکن است که در حین خواندن یا نوشتن در فایل، سیستم عامل ویندوز اجازه این کار را ندهد. در این شرایط، دچار استثنا می شویم که لازم است توسط ساختار try-except آن را مدیریت کنیم.

پاک کردن فایل

برای پاک کردن فایل باید ماژول os را در ابتدای برنامه import کنیم. توجه کنید که ماژول os، جزو ماژول های استاندارد در پایتون است.

```

File Edit Format Run Options Window Help
1 import os
2 if os.path.exists("d:\\amir\\myFile.txt"):
3     os.remove("d:\\amir\\myFile.txt")
4 else:
5     print("The file does not exist")
6

```

استفاده از متد remove جهت پاک کردن فایل

۱) تابعی بنویسید که ولتاژ دو سر یک لامپ رشته‌ای و مقاومت آن را دریافت کرده و شدت جریانی که از آن عبور می‌کند را برگرداند. سپس آن را فراخوانی کنید.

۲) با استفاده از رابطه فیثاغورس، تابعی بنویسید که سه عدد را به عنوان اضلاع یک مثلث، دریافت کرده و تشخیص دهد که آیا یک مثلث قائم الزاویه تشکیل می‌دهند؛ یا خیر؟ سپس آن را فراخوانی کنید.

پروژه

الف) برنامه‌ای بنویسید که با اجرای آن منویی با گزینه‌های زیر ظاهر شود.

زمانی که کاربر دکمه‌های a تا d از صفحه کلید رایانه را فشار دهد، دستورهای داخل منو توسط برنامه اجرا می‌شود و به طور مداوم منو ظاهر می‌شود. هنگامی که کاربر حرف e را فشار می‌دهد، از برنامه خارج شده و منو نمایش داده نمی‌شود.

ب) با توجه به مطالبی که در پودمان الگوریتم آموخته‌اید، قبل از نوشتن برنامه، الگوریتم آن را به دو روش شبه دستور و نمودار روندنما رسم کنید.

(a) اضافه کردن عدد به لیست

(b) حذف کردن عدد از لیست

(c) نمایش کل داده‌های لیست

(d) ذخیره داده‌های موجود در لیست روی حافظه دائم رایانه

(e) خروج

راهنمایی: برای هر کدام از موارد اضافه کردن، حذف و نمایش داده‌های لیست، تابعی جداگانه بنویسید و آن را فراخوانی کنید. در پایان توابع را داخل ماژولی ذخیره کرده و آن را در برنامه خودتان import کنید.



برای آشنایی با جزئیات بیشتری در خصوص پروژه، رمزینه بالا را اسکن کنید.